

Návrh a implemetace databázových systémů

Funkční závislosti, normální formy relací

Normální formy

- aby schema relace splňovalo jisté požadavky (např. na malou redundanci, odstranění aktualizčních anomálií atd.), zavádíme tzv. **normální formy**

- první
- druhá
- třetí
- Boyce-Coddova
- (čtvrtá, pátá)

Normální formy

- nejčastěji používané normální formy (druhá, třetí) jsou definovány **pomocí funkčních závislostí**
 - pro normalizaci je zjevně výhodné, když je funkčních závislostí co nejméně
 - je dobré nejprve optimalizovat jejich množinu => minimální pokrytí

První normální forma (1NF)

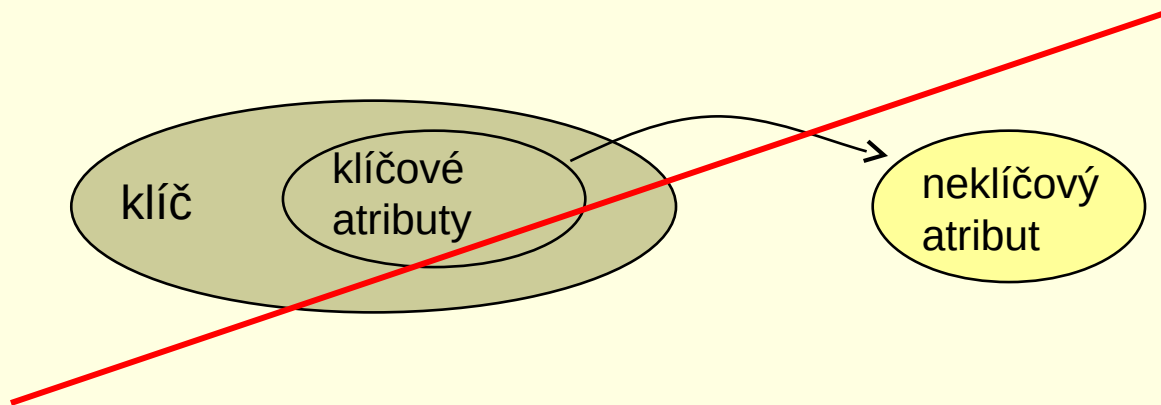
- každý atribut schématu je elementární a nestrukturovaný
 - tj. databáze je plochá, tabulka je skutečně dvojrozměrný objekt

První normální forma (1NF)

- každý atribut schématu je elementární a nestrukturovaný
 - tj. databáze je plochá, tabulka je skutečně dvojrozměrný objekt
 - OSOBA(jméno, rč, č_op) je v 1NF
 - OSOBA(jméno, rč, adresa(město, ulice, čp)) není v 1NF

Druhá normální forma (2NF)

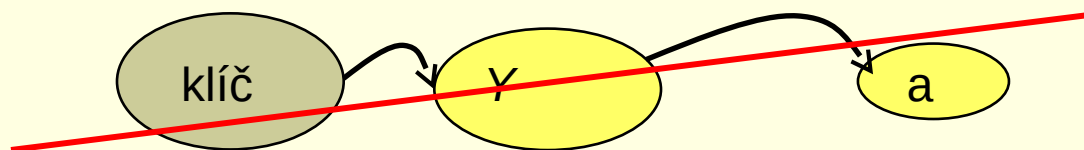
- neexistují závislosti atributů na podmnožině žádného klíče



- např. PROGRAM(název_k, jméno_f, adresa, datum)
není v 2NF, neboť platí, že $\text{název_k} \rightarrow \text{adresa}$

Třetí normální forma (3NF)

- nejdůležitější
- žádný neklíčový atribut není tranzitivně závislý na žádném klíči



- splněno, platí-li pro každou $X \rightarrow a$ (kde $X \subseteq A$, $a \in A$) aspoň jedna z podmínek
 - závislost je triviální
 - X je nadklíč
 - a je částí klíče

Třetí normální forma (3NF)

■ příklad

Firma	Sídlo	Kraj
Frantova firma	Praha	Praha
Martinova firma	Ostrava	Moravskoslezský
Pavlova firma	Brno	Jihomoravský
Viktorova firma	Praha	Praha
Pepova firma	Brno	Jihomoravský

Firma → *všechno*

Sídlo → Kraj

je ve 2NF, není ve 3NF (tranzitivní závislost Kraje na klíči přes Sídlo)

důsledek: redundance hodnot Kraje

Třetí normální forma (3NF)

■ příklad - řešení

Firma	Sídlo
Frantova firma	Praha
Martinova firma	Ostrava
Pavlova firma	Brno
Viktorova firma	Praha
Pepova firma	Brno

Firma → *všechno*

Sídlo	Kraj
Praha	Praha
Ostrava	Moravskoslezský
Brno	Jihomoravský

Sídlo → *všechno*

obě schémata jsou ve 3NF

Boyce-Coddova norm. forma (BCNF)

- každý atribut je netranzitivně závislý na klíči
- tj. pro každou netriviální závislost $X \rightarrow Y$ platí, že X obsahuje klíč schematu
- splněno, platí-li pro každou $X \rightarrow a$ (kde $X \subseteq A$, $a \in A$) aspoň jedna z podmínek
 - závislost je triviální
 - X je nadklíč

Boyce-Coddova norm. forma (BCNF)

■ příklad

Destinace	Pilot	Letadlo	Den
Paříž	kpt. Ptáček	Boeing #1	pondělí
Paříž	kpt. Ptáček	Boeing #2	úterý
Berlín	kpt. Vogel	Airbus #1	pondělí

Pilot, Den $\rightarrow$ *všechno*

Letadlo, Den $\rightarrow$ *všechno*

Destinace $\rightarrow$ Pilot

je ve 3NF, není v BCNF (Pilot závisí na Destinaci, což není nadklíč)

důsledek: redundance hodnot Pilot

Boyce-Coddova norm. forma (BCNF)

■ příklad

Destinace	Pilot
Paříž	kpt. Ptáček
Berlín	kpt. Vogel

Destinace	Letadlo	Den
Paříž	Boeing #1	pondělí
Paříž	Boeing #2	úterý
Berlín	Airbus #1	pondělí

Destinace → Pilot

Letadlo, Den → *všechno*

nyní jsou obě schémata v BCNF

Normalizace

- dva způsoby návrhu z hlediska FZ
 1. na prvotní množině relačních schemat (vzniklé např. převodem z E-R modelu) provádím normalizaci na jednotlivých schemech
 - riziko přílišného rozdrobení modelu
 2. vezmu všechny atributy všech relací a pomocí funkčních závislostí mezi nimi rozpoznávám entity
 - menší riziko rozdrobení
 - modelujeme na úrovni atributů, což není moc intuitivní
- přístupy lze různě kombinovat

Normalizace

- normalizace schematu
 - zatím jsme normálními formami řešili problém aktualizčních anomálií a redundance
 - nevhodná normalizace ale může vést k jiným potížím
 - je nutné, aby normalizovaná schemata
 - měla stejnou sémantiku (pokrytí závislostí)
 - obsahovala stejná data (bezeztrátové spojení)

Normalizace

■ pokrytí závislostí

- v původním schematu existovala množina funkčních závislostí (řekněme F)
- množina funkčních závislostí na nových schemech musí tvořit pokrytí F , tj. všechny původní FZ musí být i nadále odvoditelné

- tj. $F^+ = (\cup_{i=1..n} F_i)^+$

kde F_i je množina FZ pro dekomponované schema R_i

Normalizace

■ pokrytí závislostí - příklad

pokrytí porušeno, ztratili jsme
Sídlo → Nadmořská výška



<u>Firma</u>	Sídlo	Nadmořská výška
Sun	Santa Clara	25 mnm
Oracle	Redwood	20 mnm
Microsoft	Redmond	10 mnm
IBM	New York	15 mnm

<u>Firma</u>	Nadmořská výška
Sun	25 mnm
Oracle	20 mnm
Microsoft	10 mnm
IBM	15 mnm

Firma

<u>Firma</u>	Sídlo
Sun	Santa Clara
Oracle	Redwood
Microsoft	Redmond
IBM	New York

Firma

Firma,
Sídlo → Nadmořská výška



pokrytí zachováno

<u>Firma</u>	Sídlo
Sun	Santa Clara
Oracle	Redwood
Microsoft	Redmond
IBM	New York

Firma

<u>Sídlo</u>	Nadmořská výška
Santa Clara	25 mnm
Redwood	20 mnm
Redmond	10 mnm
New York	15 mnm

Sídlo

Normalizace

- bezeztrátová dekompozice
 - původní relační schema (před dekompozicí) by se mělo dát zrekonstruovat z dekomponovaných schemat pomocí přirozeného spojení
 - pro každou přípustnou relaci R' by mělo platit

$$R' = \ast_{i=1..n} R'[A_i],$$

kde A_i jsou atributy dílčích relací po dekompozici

- tedy spojení projekcí původní relace na množiny atributů nových relací musí vést k původní relaci

Normalizace

■ bezeztrátová dekompozice – negativní příklad

<u>Firma</u>	<u>Používá DBMS</u>	<u>Spravuje dat</u>
Sun	Oracle	50 TB
Sun	DB2	10 GB
Microsoft	MSSQL	30 TB
Microsoft	Oracle	30 TB

Firma, Používá DBMS



<u>Firma</u>	<u>Používá DBMS</u>	<u>Firma</u>	<u>Spravuje dat</u>
Sun	Oracle	Sun	50 TB
Sun	DB2	Sun	10 GB
Microsoft	MSSQL	Microsoft	30 TB
Microsoft	Oracle		

Firma, Spravuje dat

Firma, Používá DBMS

<u>Firma</u>	<u>Používá DBMS</u>	<u>Spravuje dat</u>
Sun	Oracle	50 TB
Sun	Oracle	10 GB
Sun	DB2	10 GB
Sun	DB2	50 TB
Microsoft	MSSQL	30 TB
Microsoft	Oracle	30 TB



„rekonstrukce“
(přirozené spojení)

Firma, Používá DBMS, Spravuje dat

Normalizace

- jak zajistit bezeztrátovou dekompozici?
 - mějme schema $R(A,B,C)$ (A, B, C jsou disjunktní množiny atributů) a funkční závislost $B \rightarrow C$. Pak dekompozice na $R_1(B,C)$ a $R_2(A,B)$ je bezeztrátová.
 - naopak, je-li dekompozice $R_1(B,C)$ a $R_2(A,B)$ bezeztrátová, musí platit buď $B \rightarrow C$ nebo $B \rightarrow A$.
- dekomponujeme tedy podle kritické funkční závislosti

Normalizace

■ normalizace – výhody

- lepší uložení dat v databázi, omezuje redundanci
- snižuje riziko nekonzistence

■ nevýhody

- zvyšuje četnost operace spojení, která je paměťově i časově náročná, proto někdy raději volíme nenormalizované relace kvůli rychlejší odezvě
- může značně znepřehlednit databázi
- někdy normalizací sice vzniknou relace ve 3NF, ale dojde ke ztrátě některých závislostí mezi daty, je třeba obezřetnosti
- => někdy při fyzickém návrhu volíme naopak **denormalizované tabulky** (viz Databáz. syst. 1)